Exercises
Computational Intelligence Lab
SS 2026

Machine Learning Institute
Dept. of Computer Science, ETH Zürich
Prof. Dr. Valentina Boeva
Anuj Srivastava, asrivastav@ethz.ch

Exercise sheet 11
(Optimization and Convolutional NNs)

Exercise 1 Convexity, Smoothness and Optimization

In this problem, we will investigate the convexity, smoothness, and convergence properties of the following function:

$$f(x) = ax^2, \quad a > 0. \tag{1}$$

Definition (Convexity). A function $f : \mathbb{R}^n \to \mathbb{R}$ is convex if

$$f(\lambda \mathbf{x} + (1-\lambda)\mathbf{y}) \leq \lambda f(\mathbf{x}) + (1-\lambda) f(\mathbf{y}), \quad \forall \mathbf{x}, \mathbf{y} \in \mathbb{R}^n, \lambda \in [0,1]. \tag{2}$$

Exercise 1.1 Show that the function defined in Equation (1) is convex.

Solution:

We need to show that the function in Equation (1) satisfies Equation (2). Plugging in the function, we need to show that $\forall x, y \in \mathbb{R}, \lambda \in [0,1]$, we have

$$\begin{aligned} a\big(\lambda x + (1-\lambda)y\big)^2 &\overset{?}{\leq} \lambda a x^2 + (1-\lambda) a y^2 \\ a\big(\lambda^2 x^2 + 2\lambda(1-\lambda)xy + (1-\lambda)^2 y^2\big) &\overset{?}{\leq} \lambda a x^2 + (1-\lambda) a y^2 \\ \lambda^2 x^2 + 2\lambda(1-\lambda)xy + (1-\lambda)^2 y^2 &\overset{?}{\leq} \lambda x^2 + (1-\lambda) y^2. \end{aligned}$$

Collecting all terms on the right-hand side, we have

$$\begin{aligned} 0 &\overset{?}{\leq} \lambda x^2 - \lambda^2 x^2 + (1-\lambda) y^2 - (1-\lambda)^2 y^2 - 2\lambda(1-\lambda)xy \\ &= x^2(\lambda - \lambda^2) + y^2\big((1-\lambda) - (1-\lambda)^2\big) - 2\lambda(1-\lambda)xy \\ &= x^2(\lambda - \lambda^2) + y^2(\lambda - \lambda^2) - (\lambda - \lambda^2) 2xy \\ &= \big(\lambda - \lambda^2\big)\big(x^2 + y^2 - 2xy\big) \\ &= \lambda(1-\lambda)\big(x - y\big)^2, \end{aligned}$$

which holds, because all terms in the multiplication are positive.

Definition (Smoothness). A function $f : \mathbb{R}^n \to \mathbb{R}$ is L-smooth if

$$\|\nabla f(\mathbf{x}) - \nabla f(\mathbf{y})\| \leq L\|\mathbf{x} - \mathbf{y}\|, \quad \forall \mathbf{x}, \mathbf{y} \in \mathbb{R}^n. \tag{3}$$

Exercise 1.2 Show that the function in equation 1 is smooth with parameter $L = 2a$.

Solution:

Let $x, y \in \mathbb{R}$. We have $f'(x) = 2ax$, thus

$$\|f'(x) - f'(y)\| = \|2ax - 2ay\| = |2a| \cdot \|x - y\| = 2a \cdot \|x - y\|.$$

Therefore, f is smooth for $L = 2a$.

Definition (Strong convexity). A function $f : \mathbb{R}^n \to \mathbb{R}$ is μ-strongly convex if

$$f(\mathbf{y}) \geq f(\mathbf{x}) + \langle \nabla f(\mathbf{x}), \mathbf{y} - \mathbf{x} \rangle + \frac{\mu}{2}\|\mathbf{x} - \mathbf{y}\|^2, \quad \forall \mathbf{x}, \mathbf{y} \in \mathbb{R}^n. \tag{4}$$

Exercise 1.3 Show that the function in Equation (1) is strongly convex with parameter $\mu = 2a$.

Solution:

Let $x, y \in \mathbb{R}$. Recall that $f'(x) = 2ax$. Plugging this and $\mu = 2a$ into Equation (4), we have

$$\begin{aligned} ay^2 &\overset{?}{\geq} ax^2 + 2ax(y - x) + \frac{2a}{2}(x - y)^2 \\ &= ax^2 + 2axy - 2ax^2 + a(x - y)^2 \\ &= 2axy - ax^2 + a(x - y)^2 \\ a\left(x^2 + y^2 - 2xy\right) &\overset{?}{\geq} a(x - y)^2 \\ &= a\left(x^2 + y^2 - 2xy\right). \end{aligned}$$

Thus, we have equality.

Exercise 1.4 Compute the number of gradient descent steps to find the optimum the function in Equation (1).

Solution:

The gradient descent step is

$$x_{t+1} = x_t - \eta f'(x_t).$$

Remember from the lecture that the optimal step size is $\eta = 1/L$ for an L-smooth function. Thus, for our case, the gradient descent step is

$$x_{t+1} = x_t - \frac{2ax_t}{2a} = x_t - x_t = 0.$$

Thus, at any initialization point, we have $x_1 = 0$, which is the optimum. Therefore, the maximum number of steps is 1.

Exercise 2 Stochastic Gradient Descent

In this problem, we will investigate how gradient descent converges. Recall that gradient descent iteratively makes the following step:

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \eta \nabla f(\mathbf{x}_t), \quad \eta > 0.$$

Exercise 2.1 Show that for the choice of $\eta = 1/L$, gradient descent converges to optimum of a μ-strongly convex, L-smooth function at an exponentially fast rate.

Solution:

Recall from the lecture that a μ-strongly convex implies the PL-Condition with the same μ. (*Remark*: PL condition *need not* imply strong convexity! There are non-convex functions for which PL condition holds.) Now that we have shown that our function satisfies PL-condition and the fact that it is L-smooth is given, we may deduce that it converges with a geometric rate.

Exercise 2.2 Vanilla gradient descent might not necessarily work well for many problems. We discussed a few ideas in the lecture such as adaptivity and momentum. Explain these ideas. Further comment on the potential problems that could arise from momentum.

Solution:

Momentum: We have learned that the gradients in the vicinity of stationary points are small in magnitude. This slows down the convergence of vanilla gradient descent. Momentum is a technique that has been proposed to fix this issue by increasing the effective step size by leveraging the function's smoothness. This is done by computing the direction of change from the previous two iterates and adding a scaled version to standard gradient descent:

$$\theta^{t+1} = \theta^t - \eta \nabla \ell(\theta^t) + \beta(\theta^t - \theta^{t-1}), \quad \beta \in (0, 1).$$

Adaptivity: In compositional models with billions and even trillions of parameters, it is rarely the case that one single learning rate works well for all the parameters in the model. Therefore adaptivity is a strategy to adjust the learning rate for every single parameter in the network independently. In order to adjust the learning rate for each parameter, past estimates of the gradients are stored for each parameter. The learning rate is then adjusted using the inverse of this estimate:

$$\gamma_i^t = \gamma_i^{t-1} + \left[\partial_i \ell(\theta^t)\right]^2, \quad \partial_i \ell = \frac{\partial \ell}{\partial \theta_i}$$
$$\theta_i^{t+1} = \theta_i^t - \eta_i^t \partial_i \ell(\theta^t), \quad \eta_i^t = \frac{\eta}{\sqrt{\gamma_i^t} + \delta}$$

This results in a higher learning rate for parameters with small gradients and a smaller learning rate for parameters with large gradients.

Problems with momentum: Momentum is sensitive to the choice of β that we have mentioned above. A very large value can lead to instabilities in the optimization and oscillations making convergence hard. Therefore β is usually chosen between $[0.9, 0.95]$ which seems to work very well in practice.

Exercise 2.3 Explain the differences between Gradient Descent, AdaGrad, Adam and RMSProp.

Solution:

AdaGrad:

$$\gamma_i^t = \gamma_i^{t-1} + \left[\partial_i \ell(\theta^t)\right]^2, \quad \partial_i \ell = \frac{\partial \ell}{\partial \theta_i}$$
$$\theta_i^{t+1} = \theta_i^t - \eta_i^t \partial_i \ell(\theta^t), \quad \eta_i^t = \frac{\eta}{\sqrt{\gamma_i^t} + \delta}.$$

In AdaGrad γ_i^k can grow arbitrarily large. To solve this issue RMSProp adds a decay α to the calculation of it.

RMSProp:

$$h_i^t = \alpha h_i^{t-1} + (1-\alpha)\left[\partial_i \ell(\theta^t)\right]^2, \quad \partial_i \ell = \frac{\partial \ell}{\partial \theta_i}$$
$$\theta_i^{t+1} = \theta_i^t - \frac{\eta}{\sqrt{h_i^t} + \delta} \partial_i \ell(\theta^t).$$

Notice both RMSprop and AdaGrad aims creating adaptive learning rates for the parameters but not momentum. Adam is the abbreviation for adaptive momentum estimation and is formulated as follows.

Adam:

$$h_i^t = \alpha h_i^{t-1} + (1-\alpha)\left[\partial_i \ell(\theta^t)\right]^2, \quad \partial_i \ell = \frac{\partial \ell}{\partial \theta_i}$$
$$m_i^t = \beta m_i^{t-1} + (1-\beta)\left[\partial_i \ell(\theta^t)\right]$$
$$\theta_i^{t+1} = \theta_i^t - \frac{\eta}{\sqrt{h_i^t} + \delta} m_i^t$$

Exercise 3 Optimization with Autograd (Coding)

Consider the following polynomial functions:

$$f(x) = 2x^2 - x + 1, \quad g(x) = \frac{1}{100}(x-5)(x-2)x(x+1)(x+5). \tag{5}$$

Exercise 3.1 Implement these functions in PyTorch, and plot both the function y and the gradient $\nabla g(x)$ on the interval $[-5, 5]$ using Autograd.

Solution:

See notebook.

Exercise 3.2 Find the minima of the function using Adam, Adagrad, and SGD with initial value -4 for the first polynomial and -3 for the second polynomial and print the iterate value after 20 iterations. Are 20 iterations enough to converge? Does every optimizer converge? Why, or why not?

Solution:

See notebook.

Exercise 3.3 What happens with a smaller learning rate or step size? What happens if you increase the learning rate? Plot the different results and develop a qualitative understanding.

Solution:

Experiment with the notebook.

Exercise 4 Convolutional Neural Networks

In this exercise we consider a convolutional neural network (CNN) with one convolutional layer, followed by a non-linearity ReLU, and then followed by a 3×3 max-pooling with stride 1, on top of which we apply a softmax. Assume that this small CNN takes as input an image $\mathbf{I}$ with 2 channels. Further, assume that the convolutional layer has 2 filters. Each filter consists of an odd number of pixels, such that we can index it with pixels (i, j) for $-k \leq i, j \leq k$. Moreover, assume that convolutional and max-pooling layers are performed with zero-padding, *i.e.*, by completing $\mathbf{I}$ with zeros in such a way that the convolution of $\mathbf{I}$ by a filter has the same size as $\mathbf{I}$.

Exercise 4.1 Write down the mathematical expression of the convolutional layer.

Solution:

Let $\mathbf{I}$ be an image with dimensions $M \times N \times 2$, because we have a stride 1 and a kernel size of 3, we need to pad I to be $(M+2) \times (N+2) \times 2$ by padding with a 1 line of zeros from all sides. Then the resulting $M \times N \times 2$ image I_{out} elements can be found via the equation:

$$(\mathbf{I}^{\text{padded}} \star \mathbf{K}^{\ell})_{i',j'} = \sum_{1 \leq c \leq 2} \sum_{-k \leq i,j \leq k} (\mathbf{I}_c^{\text{padded}})_{i'+i,j'+j} (\mathbf{K}_c^{\ell})_{i,j}$$

Here, the spatial dimensions were indexed by $1 \leq i' \leq M, 1 \leq j' \leq N$, and the channel dimension is indexed by $c = 1, 2$. Here $(\mathbf{I}_c^{\text{padded}})_{i'+i,j'+j} = 0$, if the indices go out of bounds, *i.e.*, $i' + i \geq M + 2$ or $j' + j \geq N + 2$

Exercise 4.2 Write down the mathematical expression of the ReLU non-linearity and the max-pooling. What is the result by composing it with the previous question? Note that this is the signal just before the softmax.

Solution:

ReLU

The ReLU activation is defined as $\text{ReLU}(x) = \max(x, 0)$. Applying such an activation function to the image is the same as applying it to each individual pixel. Thus we can combine it with the previous equation to get.

$$\mathbf{I}_{i',j'}^{\text{ReLU}} = \text{ReLU}\left((\mathbf{I}^{\text{padded}} \star \mathbf{K}^{\ell})_{i',j'}\right) = \left(0, \sum_{1 \leq c \leq 2} \sum_{-k \leq i,j \leq k} (\mathbf{I}_c^{\text{padded}})_{i'+i,j'+j} (\mathbf{K}_c^{\ell})_{i,j}\right)$$

Max-Pooling

Remember we need to again apply the padding to $\mathbf{I}^{\text{ReLU}}$ before the max-pooling. Otherwise a kernel size of 3 and stride 1 will result in dimension reduction.

$$\mathbf{I}^{\text{Max-Pool}}_{i',j'} = \max_{-1\le i,j\le 1} (\mathbf{I}^{\text{paddedReLU}})_{i+i',j+j'}$$

In the following questions, consider an image with 2 channels $\mathbf{I} = [\mathbf{I}_c]_{1\le c\le 2}$, where each $\mathbf{I}_c$ is a 4×4 matrix. Assume that each of the two filters $\mathbf{K}^\ell$ for $\ell = 1$ or $\ell = 2$ is a 2-channel 3×3 matrix per channel. We choose the image to have the following pixel values given by $\mathbf{I}_{c,i,j} = ij + c$, and the filters are defined as follows:

$$\mathbf{K}^\ell_c = \begin{bmatrix} 0 & \ell & 0 \\ \ell & -c & \ell \\ 0 & \ell & 0 \end{bmatrix}, \tag{6}$$

where $c \in \{1, 2\}$ is the channel and $\ell \in \{1, 2\}$.

Exercise 4.3 Compute the two 4×4 1-channel convolved images $\mathbf{I} \star \mathbf{K}^\ell$ for $\ell \in \{1, 2\}$. (Computing all the coefficients might be very tedious—try to do at least the first 4 ones, the top left corner.)

Solution:

We will only compute top 4 coefficients for $\ell = 1$.

$$(\mathbf{I}^{\text{padded}} \star \mathbf{K}^1)_{1,1} = (-2 + 3 + 3) + (-6 + 4 + 4) = 6$$

Similarly,

$$(\mathbf{I}^{\text{padded}} \star \mathbf{K}^1)_{1,2} = (-3 + 2 + 4 + 5) + (-8 + 3 + 5 + 6) = 14$$
$$(\mathbf{I}^{\text{padded}} \star \mathbf{K}^1)_{2,1} = (-3 + 2 + 4 + 5) + (-8 + 3 + 5 + 6) = 14$$
$$(\mathbf{I}^{\text{padded}} \star \mathbf{K}^1)_{2,2} = (-5 + 3 + 3 + 7 + 7) + (-12 + 4 + 4 + 8 + 8) = 27$$

Exercise 4.4 Write down the result of applying non-linearity ReLU to each of the convolved images.

Solution:

Since all the coefficients we found are positive applying ReLU has no effect.

Exercise 4.5 Apply the 3×3 max-pooling to each of the two images obtained in the previous question. (Compute at least the first coefficient in the top left corner.)

Solution:

The first coefficient in the top left corner is:

$$\mathbf{I}_{1,1}^{\text{Max-Pool}} = \max_{-1 \leq i,j \leq 1} (\mathbf{I}^{\text{paddedReLU}})_{i+1,j+1} = \max(6, 14, 14, 27) = 27$$